Model Loading!

Okay so we want to load models.

Firstly, let's start understanding what models are. When we want to create complex shapes beyond our typical 3D stuff, we start needing to rely on something called a model. A model is created by artists on softwares such as Blender, Maya, etc. What it essentially is, is a nice little black box of tiny triangles that have all the data of said shape saved, i.e: vertex locations, normals, texture coordinates, and sometimes colours, all exported into a file like .obj, .fbx, .gltf, or .ply. Makes it much easier to work with complex shapes without needing to code/design them vertex-by-vertex ourselves.

Library used: ASSIMP

There are many types of models, because we have a lot of artists. One of the libraries that allows us to handle all of them is Assimp. It reads basically any model format and hands it back to us as one unified structure, so our code doesn't need to care whether the file was an .obj or a .gltf. learnopengl.com/Model-Loading/Assimp

Since it no longer has any binary files available for Windows, a small workaround was done to load it into the template itself: we build Assimp from source using Visual Studio's own CMake support, then copy the resulting include/, .lib, and .dll into the project's External/assimp/ folder, same pattern as GLAD.

Mesh loaders!

Moving on, since models are technically meshes (usually several meshes glued together, like the backpack model: the bag is one mesh, the straps another, the hatchet its own), we've implemented a mesh class header that tells the code how to handle the mesh. learnopengl.com/Model-Loading/Mesh

Mesh (mesh.h) holds one piece of drawable geometry: its own vertices, indices, and textures. It sets up its own VAO/VBO/EBO, and its Draw() binds whichever textures it owns before calling glDrawElements.

Then a model class, which is there to mainly handle the vertices, indices and the texture mapping for the model and to draw it. Model (model.h) is the one that talks to Assimp directly: it parses the file, walks the node tree, and for every mesh it finds, converts Assimp's data into our own Vertex struct and builds a Mesh out of it. It also caches textures so the same image doesn't get loaded twice.

Model Rendering

To draw a model all you need to do is add this line for all your models right before the render loop. Make sure it has the right path and the right object type.

```
//you can load any model type, just ensure you set it up as a model
Model ourModel("models/backpack/backpack.obj"); // swap for your own model's path
```

https://learnopengl.com/Model-Loading/Model

Now you have it loaded, but how do we draw you ask?
You just add the code in your render loop. The transformation present here exist to show how we'd add them to our lovely model and then we draw it using our beautifully constructed shader program.

```
//to move the model weee
glm::mat4 model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(1.0f));
ourShader.setMat4("model", model);

//drawing the model
ourModel.Draw(ourShader);
```